



ENGINEERING & PROJECT CONTEXT – HANDOFF DOCUMENT

Nizalo

A global skill duel platform — one account, one rating ecosystem, one wallet, one matchmaking core, one tournament engine, one trust & safety layer, with many short-form 1v1 games behind it.

Repository	github.com/tawwerni-netizen/NeeZaaLoo
Owner	universebrands.org@gmail.com
Document date	2026-09-12
Purpose	Complete technical and product context so any engineer or AI model can pick up this work with full understanding — architecture, current build status, live deployment state, and the rules that govern every decision.

CONTENTS

Table of contents

01 What this is, and what it is not

02 The rules that govern every decision

03 System architecture

04 The Skill Duel Engine & plugin contract

05 Repository structure

06 Money: the ledger

07 Payments

08 Anti-cheat & fair play

09 Compliance & trust

10 Security & threat model

11 Build status — what is proven, phase by phase

12 Live production deployment — current state

13 Running and testing locally

14 Open decisions and next actions

15 Where to find more

What this is, and what it is not

A **global skill duel platform**: one account, one rating ecosystem, one wallet, one matchmaking core, one tournament engine, one trust & safety layer — with many short-form 1v1 games behind it. **Chess is the flagship game. Chess is not the category.**

It is not a chess website, and it is not a betting product. Those two sentences constrain every decision downstream: the name, the homepage, the information architecture, the colour palette, and the copy.

The emotional message is never "deposit money and gamble." It is: *"I want to test my skill" → "I can compete" → "I can improve" → "I can climb" → and, only where legally permitted, "I can compete for prizes."*

The product in one architectural sentence

A **Skill Duel Engine** with pluggable games sits on top of a shared platform (identity, matchmaking, rating, wallet, ledger, tournaments, risk, fair play, compliance), so that **adding a new game never requires rebuilding financial infrastructure**. If a future engineer has to touch the ledger to ship a new game, the architecture has failed.

Game roadmap and the veto rule

PHASE	GAMES
1 — Flagship	Chess
2	Speed Math · Memory Grid · Pattern · Number Memory · Reaction · Precision
3 — Evaluated / free-play only	Connect Four · Checkers · Reversi · Backgammon · Seega · Gomoku · Dominoes · XO

No game ships because it is popular. Every candidate is scored on a twelve-criterion scorecard; a low score on *cheating risk* or *server authority* is an absolute veto no total can override. That veto already excludes Connect Four, Checkers, and Reversi from cash play — Connect Four is strongly solved, so a cash game on it transfers money from honest players to whoever downloads the solver first.

Business model

Configurable rake: standard **10–15%**, VIP **8–10%**, tournaments custom — driven entirely by the Economy Rules Engine, never hardcoded. No profit is promised or projected as guaranteed; the economics only work if the platform is trusted, which is why anti-cheat, the ledger, and compliance are core product, not overhead.

The four things that would end this company

- 1 **A broken ledger** — if we cannot say what we owe, nothing else matters.

2 **Loss of custody** — a drained hot wallet.

3 **Loss of competitive integrity** — the day cheating becomes common knowledge.

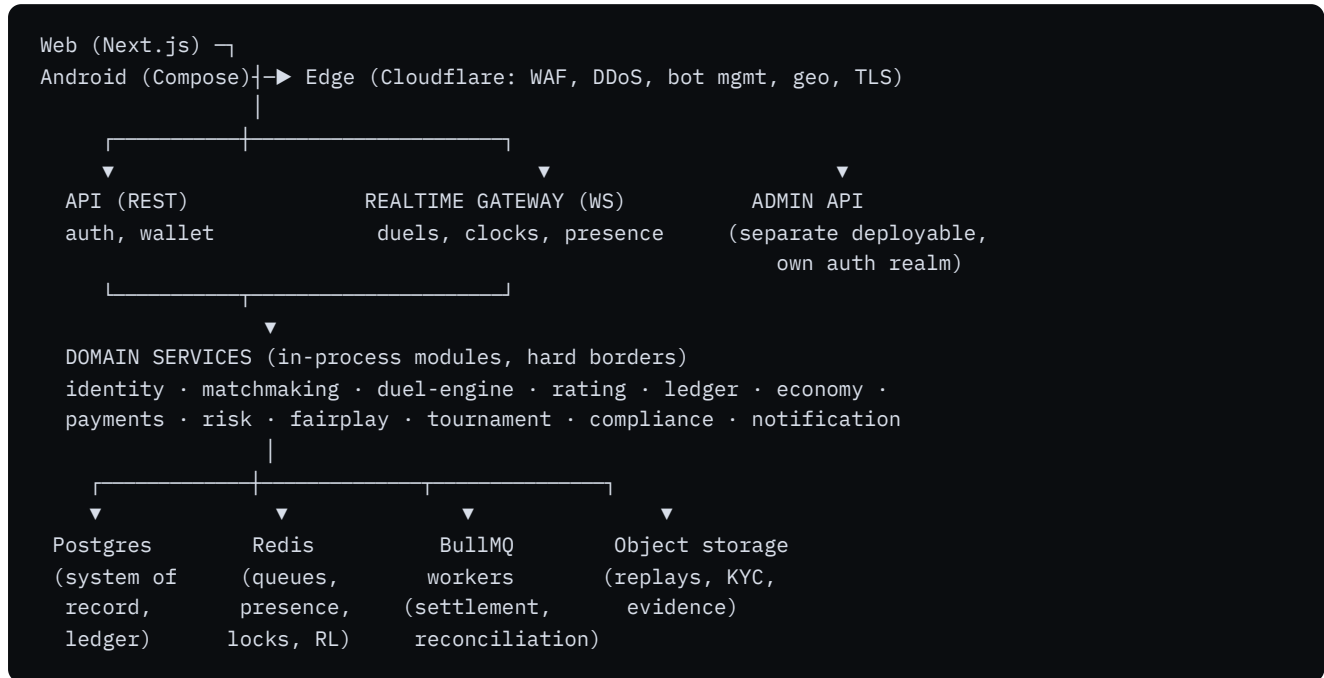
4 **Operating where not permitted** — a legal, not an engineering, failure.

The rules that govern every decision here

- 1 **The client renders; the server decides.** Nothing client-asserted is trusted for money, results, scores, state, clocks, or ratings.
- 2 **A mutable balance is never the source of truth.** Double-entry, append-only; balances are derived from entries.
- 3 **A webhook is a notification to go look, never a reason to credit.**
- 4 **Never one signal = ban.** Signals → score → evidence → case → review → decision → appeal.
- 5 **No unexplained discrepancy is ever auto-repaired.**
- 6 **Adding a game must never require touching financial infrastructure.**
- 7 **No secret in any client. No private key in any repository or APK.**
- 8 **Compliance is configuration, not conditionals.** Every market defaults to free-play-only until counsel says otherwise.
- 9 **The money surfaces feel like a bank. The game surfaces feel like a sport.** Nothing ever feels like a casino floor.

System architecture

Governing principle: the client renders; the server decides.



Why realtime is a separate deployable: game sockets are bursty, long-lived, and stateful. The money path is low-throughput and must never be starved by a chess rush hour, and a socket gateway crash must never take REST or settlement with it.

Why admin is a separate deployable: admin holds the most dangerous capabilities in the system. It gets its own deployable, its own auth realm, its own network exposure, and its own audit stream — an RCE in the public API must not be an RCE in the admin plane.

Server authority — the non-negotiable list

CONCERN	AUTHORITY	ENFORCEMENT
Money, balance	Postgres ledger	Derived from entries, never stored as truth
Game result	Duel engine (server)	Client sends <i>intents</i> ; server produces <i>events</i>
Score, state, clock	Server / game plugin	Client holds a projection it may not author
Rating	Rating service	Computed post-settlement from server's own result
Payment / withdrawal	Payment service + ledger	Only from verified provider/chain state; never client-advanced

Every client→server game message is an **intent** (SubmitMove, Resign); every server→client message is an **event** (MoveAccepted, DuelSettled). There is no message shape in which a client asserts a result — enforced at the protocol schema level.

The Skill Duel Engine & plugin contract

```
SkillDuelEngine (owns: lifecycle, clock, persistence, events, settlement)
├─ ChessPlugin          └─ DominoesPlugin
├─ SpeedMathPlugin      └─ BackgammonPlugin
├─ CheckersPlugin       └─ SeegaPlugin
├─ ConnectFourPlugin    └─ ReversiPlugin
└─ XOPlugin             └─ GomokuPlugin
```

The engine owns everything touching money, identity, or trust. A plugin owns only the rules of its game. Four properties the contract buys: **purity** (pure functions, fully unit-testable, exact replay verification), **server-generated challenges** (the seed is never sent to the client before content is due), **projection** (`project()` guarantees a client never receives data it should not have — if it's not in the projection, it's not on the wire), and **signals not verdicts** (a plugin can report an anomaly; it cannot ban anyone — enforced by the type system).

Duel lifecycle

```
CREATED → RESERVED → READY → LIVE → COMPLETED → SETTLED
                        ↘ ABORTED / VOIDED
```

RESERVED: entry fees move to a locked ledger position before the duel starts — no duel with money attached ever begins with unlocked funds. **COMPLETED vs SETTLED** are deliberately distinct steps, so a settlement failure can be retried without re-running the game, and a duel under fair-play hold can sit in COMPLETED indefinitely.

Game selection scorecard

Weighted 12-criterion score; a game needs $\geq 70\%$ of maximum **and** $\geq 4/5$ on both cheating-risk and server-authority to enter development — a high total cannot compensate for a low cheat-risk score.

GAME	CHEAT RISK	SERVER AUTHORITY	VERDICT
Chess	2/5	5/5	Ship. Flagship; heaviest anti-cheat investment.
Speed Math / Memory / Pattern	4/5	5/5	Ship.
Reaction / Precision	3/5	4/5	Ship with caution. Latency-oracle risk — consider free-play only.
Connect Four / Checkers / Reversi	1/5 — solved/near-solved	5/5	Never cash. Free-play only.

Repository structure

npm workspaces monorepo. H:\Claide Apps\nizalo · GitHub: tawwerni-netizen/NeeZaaLoo, branch main. Root package.json is "type": "commonjs" (required for the Hostinger Node-20 preload system); every app and package underneath keeps its own "type": "module".

4 APPS	37 PACKAGES	46 DB MIGRATIONS	10 GAMES SHIPPED
------------------	-----------------------	----------------------------	----------------------------

PATH	WHAT IT IS
apps/api	REST API server. Auth, wallet, tournaments, admin surface. <code>src/index.mjs</code> is the entry point.
apps/gateway	Realtime WebSocket gateway — duels, clocks, presence, chat. Separate deployable from the API by design (see §3).
apps/worker	Background workers — settlement, reconciliation, tournament sweeps, matchmaking dispatch.
apps/web	Next.js 16 (Turbopack) frontend, App Router, output: "standalone", i18n (en/zh/hi/ar+2 more locales).
packages/ledger	Double-entry primitives, migration runner (<code>migrate.mjs</code>), Postgres adapter with deadlock retry.
packages/game-*	10 plugin packages — chess, checkers, connect-four, xo, speed-math, dominoes, backgammon, seega, reversi, gomoku. Each exports a Plugin + an AI adapter.
packages/auth / authz	Argon2id auth, TOTP 2FA, session/device management · deny-by-default RBAC (9 admin roles).
packages/tournament / mastery / engagement / replay	8-phase tournament lifecycle · non-inflatable per-game mastery levels · streaks/daily-challenges/recommendations · on-demand replay rebuilding.
packages/fairplay / compliance / reconciliation	Fair Play Engine (signals→score→case) · market matrix policy layer · L1/L2/L3 + solvency reconciliation.
packages/tokens	Design tokens, both themes, full RTL — <code>tokens.css</code> .
db/migrations/	46 forward-only, numbered SQL files, tracked in a <code>schema_migration</code> table. No <code>CREATE EXTENSION</code> — functions like <code>gen_random_uuid()</code> must pre-exist on the target DB.
brand/	12 SVG identity assets + the brand guidelines document (this PDF's companion).

The ledger

Every transaction sums to zero, per asset. Therefore the system always balances, or it refuses the write. Balances are derived by summing entries; a mutable balance column is never the source of truth.

Account model

Wallet states are **distinct accounts**, not a status flag on one balance — "locked" funds live in a different account, so a query for available balance cannot accidentally include them.

ACCOUNT	MEANING
user:{id}:available	Spendable now
user:{id}:locked	Committed to a live duel or pending withdrawal
user:{id}:pending	Deposit seen, not yet finalised
user:{id}:withdrawable	Cleared, eligible for withdrawal
platform:suspense	The honesty account. Unexplained funds land here and raise a case — never auto-cleared.

Enforced invariants (all at the database layer)

I1	Every transaction sums to zero per asset	Deferred constraint trigger
I2	Entries are immutable	Revoked UPDATE/DELETE + trigger
I3	No user account goes negative	Checked under row lock
I4	Idempotency keys are unique	Unique index — a retry cannot double-post
I5	Adjustments require a reason + admin actor	CHECK constraint

Reconciliation runs continuously at three levels — L1 internal (snapshot vs. entries), L2 provider (our records vs. NOWPayments), L3 chain (custody vs. actual on-chain balance) — plus a global solvency check: $\text{SUM}(\text{custody}) \geq \text{SUM}(\text{liabilities})$ per asset. A failure triggers an immediate automated withdrawal halt and pages a human. Discrepancies always produce **ALERT** → **CASE** → **REVIEW**, never a silent repair.

Payments

MVP asset: USDT · **MVP network:** TRON (TRC20), one network only · **Provider:** NOWPayments, behind a `PaymentProvider` abstraction the core platform never sees past.

A webhook is a notification to go look. It is never the reason we credit. Credit happens only after independent on-chain verification of asset, network, amount, destination, and confirmation depth.

Withdrawal state machine

```
REQUESTED → VALIDATING → RISK_CHECK → PENDING_REVIEW → APPROVED  
→ PROCESSING → BROADCASTED → CONFIRMED → COMPLETED  
(or FAILED | REJECTED | CANCELLED)
```

Funds lock at REQUESTED. No transition is ever triggered by a client call. **Four-eyes approval** above a configured threshold — no single admin can both initiate and approve. Nine gates must all pass: balance, KYC, geo, age, address allowlist, security cooling-off, risk score, fair-play hold, and global solvency.

Custody

Unresolved, blocks Phase 5 (founder + counsel decision). Options on the table: full provider custody, qualified third-party custodian, or MPC/HSM self-custody. Whatever is chosen: no private key in source code or the APK, ever; hot-wallet float capped and topped up from cold storage on a schedule; signing is a separate, minimal-surface service.

Anti-cheat & fair play

Governing rule: never one signal = ban. Signals → Risk Score → Evidence → Case → Review → Decision → Appeal. The plugin contract enforces this structurally — `fairPlaySignals()` returns signals and has no type-level way to express a verdict.

Chess-specific detection (the hardest problem in the product)

Free engines exceed every human, so detection is statistical, never certain: engine correlation, ACPL, critical-position accuracy (the most discriminating signal — humans degrade on only-move positions, engines don't), move-time distribution, rating-performance delta, longitudinal consistency, and selective assistance. **Thresholds come from labelled data, not intuition** — this dataset does not exist yet (open item A1), so chess detection currently produces cases for human review with **no automated sanctions**.

Anti-collusion

Modelled as a graph problem: nodes are accounts/devices/IPs/payment instruments; edges are shared device, shared funding, repeated pairing, value flow. Abnormal pairing frequency is computable exactly, since the platform controls matchmaking and knows the expected meeting distribution.

What may be automated vs. what always needs a human

CLASS	AUTO-ACTION?
Physically impossible input; confirmed multi-account self-play	YES
Statistical engine correlation; collusion graph anomaly; input-biometric anomaly	NO — always human

Compliance & trust

A separate policy layer every sensitive action consults — never conditionals scattered through feature code — able to change a market's status by **configuration, not deployment**. **Fail closed:** if the policy engine cannot reach a dependency, real-money actions are denied; free play continues.

Default posture

Every country starts UNDETERMINED → ALLOW_FREE_PLAY_ONLY — no real money by default. A market becomes real-money eligible only by an explicit, counsel-backed, audited change.

KYC tiers

TIER	TRIGGER	UNLOCKS
0	Registration	Free play, ranked play
1	First deposit	Cash duels within limits
2	Withdrawal / volume threshold	Withdrawals within limits
3	High volume / risk flag	Elevated limits

KYC is enforced at the withdrawal boundary at minimum — a platform that allows anonymous deposit→play→withdraw is a laundering vehicle. Documents live in separate encrypted storage with their own retention clock.

Threat model

STRIDE per trust boundary, re-run at every phase gate. **Three of the top four assets at risk are not classic security problems** — a perfectly hardened server with a working cheat is a dead company.

#	ASSET	SEVERITY
1	Ledger integrity	Terminal
2	Custody / withdrawal control	Terminal
3	Competitive integrity	Terminal
4	Regulatory standing	Terminal

Highest-value attack surface: payment provider webhooks. A forged webhook that credits a balance is a direct mint. Mitigation: HMAC verification before parsing (constant-time), idempotency on provider event id, and mandatory independent on-chain confirmation before any credit — never the webhook alone.

Everything left of the client is hostile — the Android APK will be decompiled and the WebSocket protocol reverse-engineered. There is no client-side secret and no client-side check that matters.

Standing security requirements

No secret in any client · no private keys in application code or servers · deny by default on every endpoint (unannotated endpoints fail closed in CI) · all financial mutations idempotent and audited · fail closed on money, fail open on fun.

11 — STATUS

Build status — what is proven, phase by phase

Each roadmap item is demonstrated by a test that fails if the guard is removed — "PROVEN" is not an assertion, it's a passing test suite.

PHASE	SCOPE	STATUS
0 — Foundation	Docs, threat model, ledger spec, brand, compliance & anti-cheat architecture	GATE PASSED 2026-09-06
1 — Platform core	Ledger invariants, auth (Argon2id + TOTP), authz, REST contract	PROVEN — 22/22 tests vs real Postgres 17
2 — Chess, realtime, ratings	Server-authoritative chess, Glicko-2, matchmaking, WS gateway, concurrency	PROVEN — 746/746, incl. real-Postgres concurrency (8 connections, deadlock handling)
3 — Skill Duel Engine + games	10 games shipped as plugins; server-seeded challenges	Built — chess runs unmodified as a plugin
4 — Tournaments, progression, admin	8-phase tournament lifecycle, Global Skill Score (35% cap), 9-role RBAC, mastery, streaks, replay center	Built & regression-tested — 2,122/2,122 across 37 packages
5 — Money in production	Full ledger wiring, NOWPayments sandbox, four-eyes approval	Sandbox only — production custody model (D4) still open
6 — Compliance, KYC/AML	Provider integration, market matrix enforcement, production payment keys	Not started — blocked on legal entity + counsel (D2, D3)
7 — Android	Kotlin + Jetpack Compose client	Not started
8 — Hardening	Pen-test, load/soak, DR drills	Not started
9 — Launch	Staged rollout, cash tiers per licensed market	Not started

What "PROVEN" does and doesn't mean: genuine parallelism (row-lock contention, deadlock ordering, serialisation failures) is exercised against real PostgreSQL for the specific scenarios the risk register names — real financial writes: withdrawals, reservations, settlement, deposits. Everyday tests still run single-connection PGLite; a *new* financial write path added later needs its own real-Postgres concurrency scenario, not just a PGLite test.

Live production deployment — current state

This section reflects the deployment work completed 2026-09-11/12 and is the most likely part of this document to go stale first — verify against the live Hostinger dashboards before trusting it.

Hosting: Hostinger

SITE	DEPLOYS	ROOT DIR	STATUS
nizalo.com	apps/web (Next.js, standalone build, flattened via apps/web/scripts/package-standalone.mjs)	./	Live
stage.nizalo.com	staging copy of apps/web	./	Exists, not actively used this pass
api.nizalo.com	apps/api (Node REST server)	apps/api	Deployed 2026-09-11 — build config fixed (see below)

All three auto-deploy from GitHub (tawwerni-netizen/NeeZaaLoo, branch main) on every push, via Hostinger's Git-connected "Web Apps" flow. Node runtime: **20.x** (this is why root package.json is "type": "commonjs" — Hostinger's preload system uses require() and breaks under Node 20 if the root is ESM).

API.NIZALO.COM BUILD CONFIGURATION (AS CORRECTED)

apps/api is a plain Node server with no build step. The first deploy attempt used the default npm run build, which failed (Missing script: "build" — apps/api has no such script). Fixed to: **Build command: None · Entry file: src/index.mjs** · Root directory apps/api. Redeployed successfully.

ENVIRONMENT VARIABLES SET ON API.NIZALO.COM

DATABASE_URL	Postgres connection string (see Database below)
NODE_ENV	production
AUTH_SIGNING_KEY_B64	Freshly generated 32-byte key, base64 — never reused from staging
AUTH_ENCRYPTION_KEY_B64	Freshly generated 32-byte key, base64 — never reused from staging

Both keys were generated fresh for this deployment and are not reproduced in this document. If they need rotating, generate via node -e "console.log(require('crypto').randomBytes(32).toString('base64'))" and update both Hostinger's environment variables panel and any dependent service.

Database: Supabase Postgres

Project oqauuhkztracrktplxp on Supabase (<https://oqauuhkztracrktplxp.supabase.co>). **Important distinction discovered during setup:** Supabase surfaces several different connection values that are easy to confuse —

VALUE	LOOKS LIKE	USE FOR
Project REST URL	<code>https://xxx.supabase.co/rest/v1/</code>	Supabase's own REST/client API — not a Postgres connection string
Publishable / secret API key	<code>sb_publishable_...</code> / <code>sb_secret_...</code>	Supabase's REST/management API auth — not DATABASE_URL
Postgres connection string	<code>postgresql://postgres:[PW]@db.xxx.supabase.co:5432/postgres</code>	This is DATABASE_URL — found under Project Settings → Database → Connection string → "Direct connection" → type "URI"

46 migrations have not yet been confirmed run against this production database as of this document's writing — run `packages/ledger/src/migrate.mjs's migrate(db, {dir, log:true})` against the real DATABASE_URL before relying on signup/auth working end-to-end. No `CREATE EXTENSION` statements exist in the migration set, so confirm `gen_random_uuid()` (`pgcrypto`) is available on the target — Supabase enables it by default.

Frontend ↔ API wiring

The root cause of the original "signup does nothing" production bug: `apps/web/src/lib/api.ts` defaults `NEXT_PUBLIC_API_URL` to `http://localhost:3000` when the variable isn't set at **build time** (Next.js bakes `NEXT_PUBLIC_*` vars into the JS bundle — setting them after the build has no effect). Similarly `apps/web/src/lib/use-duel-socket.ts` defaults `NEXT_PUBLIC_GATEWAY_URL` to `ws://localhost:3010`.

Required on nizalo.com's environment variables, before the next build:

<code>NEXT_PUBLIC_API_URL</code>	<code>https://api.nizalo.com</code>
<code>NEXT_PUBLIC_GATEWAY_URL</code>	<code>wss://[gateway host — apps/gateway not yet deployed as of this doc]</code>

apps/gateway (the realtime WebSocket server for live duels/chat) has not been deployed to production yet. Only `apps/web` and `apps/api` are live. Realtime play, live duel spectating, and chat will not function until `apps/gateway` is deployed to a domain/port reachable over `wss://` and `NEXT_PUBLIC_GATEWAY_URL` is wired to it — this is the next infrastructure gap after this document was written.

Known open items from this deployment pass

- Confirm DATABASE_URL on `api.nizalo.com` is the real Postgres URI (not a Supabase REST URL) and that migrations have been run against it.
- Set `NEXT_PUBLIC_API_URL=https://api.nizalo.com` on `nizalo.com` and trigger a rebuild (this variable only takes effect at build time).

- Deploy apps/gateway for realtime features; wire NEXT_PUBLIC_GATEWAY_URL.
- Deploy apps/worker (settlement, reconciliation, tournament sweeps) — not covered in this pass; without it, cash settlement and tournament automation will not run in production.
- A double-modal rendering bug was observed on the signup form (cosmetic, non-blocking).
- Any Supabase key pasted into a chat/session during this setup should be rotated as a precaution (Project Settings → API).

Running and testing locally

```
npm install
npm test          # 2,122+ tests across 37 workspaces, PGLite (Postgres 17 in WASM)
npm run typecheck
npm run build --workspace=apps/web
npm run start --workspace=apps/web
```

Tests run against **PGLite** — real PostgreSQL 17 compiled to WASM, in-process, no Docker required. This proves every constraint, trigger, and `plpgsql` function exactly as production, with one limitation: **PGLite is single-connection**, so genuine concurrency (row-lock contention, deadlocks, serialisation failures) is not exercised by the default suite. Real-Postgres concurrency scenarios live in `*/real-pg-concurrency.test.mjs` files and use `packages/ledger/test-support/real-pg-db.mjs` to provision a disposable database per test file against `TEST_DATABASE_URL` (defaults to `postgres://postgres:postgres@localhost:5432/skill_platform_test`) — a real local or containerised Postgres is required to run these.

A PGLite-socket bridge is available for standing up a full live dev environment (API + web dev server) with no local Postgres install at all — see the project memory note *"Smoke-test env via PGLite-socket"* for the exact recipe.

Open decisions and next actions

#	DECISION	BLOCKS	OWNER
D2	Legal entity, jurisdiction, licensing counsel	All real-money features (Phase 6)	Founder + counsel
D3	Launch markets (the market matrix)	Phase 6	Founder + counsel
D4	Custody model (provider / qualified custodian / MPC)	Phase 5	Founder
D5	Whether Reaction/Precision may ever be cash games	Phase 3 cash tiers	Engineering + founder
A1	Labelled dataset for chess cheat-detection tuning	Any automated chess sanction	Engineering
A2	Latency-oracle bound proven for Reaction/Precision	Cash tiers for those games	Engineering

Engineering can proceed through Phase 4 without D2–D5 resolved, and cannot proceed past it — Phase 5 (money in production) and Phase 6 (compliance/KYC) are gated on founder and legal decisions, not code.

Where to find more

This document condenses the full documentation tree at docs/. For anything requiring exact wording, cite the source file, not this summary:

docs/product/PROJECT_VISION.md	Full vision, game design philosophy, growth & psychology rules
docs/product/ROADMAP.md	All 10 phases with hard gates and exact test counts
docs/architecture/TECHNICAL_ARCHITECTURE.md	Full system design, plugin contract, matchmaking internals
docs/architecture/ENVIRONMENT_ASSESSMENT.md	Toolchain, stack decision rationale
docs/architecture/RISK_REGISTER.md	The 2026-09-06 concurrency audit (A2, A8, A9) in full
docs/security/THREAT_MODEL.md	Full STRIDE analysis, all 8 trust boundaries
docs/ledger/LEDGER_SPECIFICATION.md	Complete schema, all canonical flows
docs/payments/PAYMENT_ARCHITECTURE.md	Full deposit/withdrawal state machines
docs/anti-cheat/ANTI_CHEAT_ARCHITECTURE.md	Full signal catalogue, case management
docs/compliance/COMPLIANCE_ARCHITECTURE.md	Full market matrix schema, KYC/AML detail
docs/brand/BRAND_GUIDELINES.md	Companion document: <i>Nizalo — Brand Identity.pdf</i>
README.md	Quick start, the 9 founding rules, full document index

Generated 2026-09-12 as a handoff/reference document. Source repository is the single source of truth — this PDF will drift from it over time and should be regenerated periodically, particularly §12 (live deployment state).